

Guía de Instalación y Configuración: Java + Visual Studio Code

Materia: Fundamentos de Programación (IAD-2413) — ITD / TecNM

Objetivo: que cada alumno tenga, en menos de 30 minutos y sin depender del profesor, un entorno funcional para escribir, compilar, ejecutar y depurar programas en Java usando Visual Studio Code (VS Code).

Nota didáctica para el profesor: en Java, a diferencia de Python, el 90% de los problemas de instalación vienen de dos cosas: (1) instalar un JDK equivocado o incompleto, y (2) que el sistema no sabe dónde quedó instalado ese JDK (variable `JAVA_HOME`). Esta guía está diseñada para que ambos puntos queden resueltos antes de escribir una sola línea de código.

0. ¿Qué SDK de Java usar? (decisión importante antes de instalar)

Para programar en Java se necesita el **JDK (Java Development Kit)**, no solo el JRE (que únicamente ejecuta programas, no los compila). Existen varias distribuciones del JDK; las tres más comunes son:

Distribución	Costo / licencia	Comentario
Oracle JDK	Gratis para uso general desde JDK 17, pero con términos de licencia (NFTC) que hay que leer con cuidado, y en versiones anteriores requería suscripción para uso en producción	Es la "original", pero su licencia ha generado confusión incluso en entornos profesionales
Eclipse Temurin (de la Eclipse Adoptium Foundation)	100% gratuito, código abierto, sin letras chiquitas	Es el sucesor directo del antiguo AdoptOpenJDK, muy usado en la industria
Microsoft Build of OpenJDK	100% gratuito, código abierto, mantenido por Microsoft	Se integra de forma nativa y automática con la extensión de Java de VS Code

Recomendación para este curso: Microsoft Build of OpenJDK

Razones concretas:

1. **Cero fricción con VS Code:** la extensión oficial de Java para VS Code (que instalaremos en la Parte 2) puede descargar e instalar este JDK automáticamente, sin

que el alumno tenga que ir a buscar un instalador aparte.

2. **Sin ambigüedad de licencia:** es completamente gratuito para cualquier uso, sin necesidad de leer términos legales antes de usarlo en el laboratorio.
3. **Es un JDK estándar:** al ser una distribución de OpenJDK, el código que se escriba es 100% compatible con cualquier otra distribución (Oracle, Temurin, etc.), así que no hay riesgo de "atarse" a algo no estándar.

Si en tu instituto ya tienen Eclipse Temurin instalado en los laboratorios, es igualmente válido y esta guía funciona igual; el único cambio sería el sitio de descarga en la Parte 1.

Parte 1: Instalar el JDK

Opción A (recomendada): dejar que VS Code lo instale automáticamente

Se puede omitir esta parte por completo e instalar el JDK desde dentro de VS Code en la Parte 2, sección 2.3. Es la forma más simple para un grupo grande, porque VS Code descarga la versión correcta automáticamente.

Opción B: instalar el JDK manualmente antes de abrir VS Code

Recomendable si el laboratorio no tiene buena conexión a internet al momento de trabajar en VS Code, o si se quiere tener todo listo de antemano.

1. Ir a <https://learn.microsoft.com/java/openjdk/download>
 2. Descargar la versión **LTS más reciente** (por ejemplo, JDK 21), eligiendo el instalador para el sistema operativo correspondiente (`.msi` para Windows, `.pkg` para macOS, `.tar.gz` para Linux).
 3. Ejecutar el instalador con las opciones por defecto.
 - **Windows:** el instalador `.msi` de Microsoft Build of OpenJDK configura automáticamente la variable `JAVA_HOME` y agrega Java al PATH. Este es uno de sus grandes beneficios frente a otras distribuciones.
 4. Reiniciar el equipo (o al menos cerrar y volver a abrir cualquier terminal) para que los cambios de variables de entorno surtan efecto.
-

Parte 2: Verificar el JDK y preparar VS Code

2.1 Verificar que el JDK quedó instalado (solo si se hizo la Opción B)

Abrir una terminal (`cmd`/PowerShell en Windows, Terminal en macOS/Linux) y escribir:

```
bash
```

```
java -version
```

```
javac -version
```

- `java -version` confirma que se puede **ejecutar** código Java.
- `javac -version` confirma que se puede **compilar** código Java (`javac` es el compilador; si falta, normalmente significa que se instaló un JRE en vez de un JDK completo).

Ambos comandos deben mostrar el mismo número de versión, sin errores.

2.2 Instalar Visual Studio Code

1. Ir a <https://code.visualstudio.com/> y descargar la versión para el sistema operativo correspondiente.
2. Instalar con las opciones por defecto. En Windows, dejar marcadas:
 - ☒ "Add to PATH"
 - ☒ "Add 'Open with Code' action to Windows Explorer context menu"

2.3 Instalar el paquete de extensiones de Java

1. Abrir VS Code.
2. Ir al ícono de **Extensiones** (`Ctrl+Shift+X`).
3. Buscar "**Extension Pack for Java**" — publicado por **Microsoft**. Este es el paso más importante de toda la guía: es un solo paquete que instala automáticamente TODO lo necesario:
 - Language Support for Java (compilación, autocompletado, errores en tiempo real)
 - Debugger for Java (depuración)
 - Test Runner for Java (pruebas unitarias, útil para prácticas posteriores)
 - Maven for Java (gestión de proyectos)
 - Project Manager for Java
4. Dar clic en **Instalar**. VS Code instalará los cinco componentes de forma automática.

2.4 Si no se instaló el JDK antes (Opción A)

Al abrir el primer archivo (`.java`), VS Code detectará que no hay un JDK configurado y mostrará una notificación con un botón como "**Install a JDK**" o "**Download JDK**". Al darle clic:

1. Elegir la opción que ofrece **Microsoft Build of OpenJDK**.
2. VS Code lo descarga e instala en una carpeta interna, sin que el alumno tenga que configurar nada manualmente.

Esta es la ruta más recomendable para un salón completo, porque elimina el paso de "cada quien instala el JDK por su cuenta con posibles variantes".

Parte 3: Crear la carpeta de trabajo y el primer programa

1. Crear una carpeta de trabajo, por ejemplo `FundamentosProgramacion`.
2. En VS Code: **Archivo** → **Abrir carpeta...** y seleccionar esa carpeta.
3. Es muy recomendable dejar que VS Code cree la estructura de proyecto por nosotros:
 - Abrir la paleta de comandos (`Ctrl+Shift+P`) y escribir "**Java: Create Java Project**".
 - Elegir "**No build tools**" para un proyecto simple (ideal para las primeras prácticas; más adelante se puede usar Maven o Gradle).
 - Seleccionar la carpeta de trabajo.

Esto genera automáticamente una carpeta `src/` (donde va el código) y una `bin/` (donde van los archivos compilados) — la misma estructura que se usa en proyectos reales.

4. Dentro de `src/`, VS Code ya crea un archivo `Main.java` de ejemplo. Editarlo así:

```
java

import java.util.Scanner;

public class Main {
    public static void main(String[] args) {
        System.out.println("Hola, mundo");

        Scanner scanner = new Scanner(System.in);
        System.out.print("¿Cómo te llamas? ");
        String nombre = scanner.nextLine();

        System.out.println("Bienvenido a Fundamentos de Programación, " + nombre);
        scanner.close();
    }
}
```

Regla clave de Java que hay que remarcar a los alumnos: **el nombre del archivo debe coincidir exactamente con el nombre de la clase pública** (`Main.java` → `public class Main`). Si no coinciden, no compila, y este es uno de los errores más comunes en las primeras semanas.

5. Guardar el archivo (`Ctrl+S`).

Ejecutar el programa

- **Botón rápido:** clic en "Run", que aparece automáticamente arriba del método `main`.
- **Terminal integrada,** desde la carpeta del proyecto:

```
bash

javac src/Main.java -d bin
java -cp bin Main
```

Para el día a día en clase, conviene usar siempre el botón "Run" de VS Code, ya que compila y ejecuta en un solo paso; los comandos manuales son útiles para que los alumnos entiendan qué está pasando "por debajo".

Parte 4: Depuración básica

1. Dar clic a la izquierda del número de línea para poner un **breakpoint** (punto rojo).
2. Presionar `F5`, o dar clic en "Debug" (aparece junto al botón "Run" arriba del método `main`).
3. El programa se detendrá en el breakpoint; en el panel izquierdo se pueden inspeccionar las variables (`nombre`, `scanner`, etc.) en tiempo real.

Tabla de errores comunes y solución rápida

Síntoma	Causa probable	Solución
<code>'java'</code> no se reconoce como un comando	El JDK no quedó en el PATH / no se reinició la terminal	Cerrar y reabrir la terminal; si persiste, reinstalar el JDK verificando que configure <code>JAVA_HOME</code>
<code>javac</code> no funciona pero <code>java</code> sí	Se instaló solo un JRE, no un JDK completo	Reinstalar usando el instalador del JDK completo de Microsoft Build of

Síntoma	Causa probable	Solución
		OpenJDK
<code>class Main is public, should be declared in a file named Main.java</code>	El nombre del archivo no coincide con el de la clase pública	Renombrar el archivo para que coincida exactamente (incluyendo mayúsculas)
VS Code no muestra el botón "Run" sobre <code>main</code>	Falta el "Extension Pack for Java" o el proyecto no se creó como proyecto de Java	Instalar el Extension Pack y volver a crear el proyecto con "Java: Create Java Project"
Error <code>JAVA_HOME is not defined</code> al usar herramientas externas (Maven, etc.)	La variable de entorno no quedó configurada	En Windows: Configuración → Variables de entorno → crear <code>JAVA_HOME</code> apuntando a la carpeta de instalación del JDK
El programa no pide el <code>nombre</code> y se cierra de inmediato	Se está ejecutando desde un botón que no soporta entrada por teclado (poco común)	Ejecutar desde la terminal integrada de VS Code, que sí soporta <code>Scanner</code> / <code>input</code>

Checklist de verificación antes de la primera práctica

- `java -version` y `javac -version` responden con el mismo número de versión, sin errores.
- VS Code tiene instalado el "Extension Pack for Java" (Microsoft).
- Se creó un proyecto con "Java: Create Java Project" y aparece la estructura `src/` / `bin/`.
- El archivo `Main.java` corrió correctamente con el botón "Run" y respondió a la entrada de teclado.
- Se probó al menos un breakpoint con `F5`.

Recomendación para la sesión de clase

Para Java conviene invertir un poco más de tiempo en la instalación que con Python, precisamente por el tema del JDK y las variables de entorno. Sugerencia de dinámica: instalar en vivo usando la **Opción A** (dejar que VS Code descargue el JDK de Microsoft automáticamente al abrir el primer archivo `.java`), ya que reduce a la mitad los pasos manuales y evita que cada alumno traiga una versión distinta de JDK instalada desde antes.

Cerrar la sesión con el checklist anterior antes de avanzar al Tema 1.5 (sintaxis para definición de funciones y métodos).